JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.chrono

Class JapaneseEra

java.lang.Object
    java.time.chrono.JapaneseEra

All Implemented Interfaces:

`Serializable`, `Era`, `TemporalAccessor`, `TemporalAdjuster`

```
public final class JapaneseEra
extends Object
implements Era, Serializable
```

An era in the Japanese Imperial calendar system.

The Japanese government defines the official name and start date of each era. Eras are consecutive and their date ranges do not overlap, so the end date of one era is always the day before the start date of the next era.

The Java SE Platform supports all eras defined by the Japanese government, beginning with the Meiji era. Each era is identified in the Platform by an integer value and a name. The `of(int)` and `valueOf(String)` methods may be used to obtain a singleton instance of JapaneseEra for each era. The `values()` method returns the singleton instances of all supported eras.

For convenience, this class declares a number of public static final fields that refer to singleton instances returned by the values() method.

API Note:

The fields declared in this class may evolve over time, in line with the results of the `values()` method. However, there is not necessarily a 1:1 correspondence between the fields and the singleton instances., The Japanese government may announce a new era and define its start date but not its official name. In this scenario, the singleton instance that represents the new era may return a name that is not stable until the official name is defined. Developers should exercise caution when

relying on the name returned by any singleton instance that does not correspond to a public static final field.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static JapaneseEra	HEISEI	The singleton instance for the 'Heisei' era (1989-01-08 - 2019-04-30) which has the value 2.
static JapaneseEra	MEIJI	The singleton instance for the 'Meiji' era (1868-01-01 - 1912-07-29) which has the value -1.
static JapaneseEra	SHOWA	The singleton instance for the 'Showa' era (1926-12-25 - 1989-01-07) which has the value 1.
static JapaneseEra	TAISHO	The singleton instance for the 'Taisho' era (1912-07-30 - 1926-12-24) which has the value 0.

Method Summary

Concrete Methods **Instance Methods**

Modifier and Type	Method	Description
String	getDisplayName(TextStyle style, Locale locale)	Gets the textual representation of this era.

int	**getValue**()	Gets the numeric era int value.
static **JapaneseEra**	**of**(int japaneseEra)	Obtains an instance of JapaneseEra from an int value.
ValueRange	**range**(**TemporalField** field)	Gets the range of valid values for the specified field.
String	**toString**()	Returns a string representation of the object.
static **JapaneseEra**	**valueOf**(**String** japaneseEra)	Returns the JapaneseEra with the name.
static **JapaneseEra**[]	**values**()	Returns an array of JapaneseEras.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.time.chrono.Era

adjustInto, get, getLong, isSupported, query

Field Detail

MEIJI

```
public static final JapaneseEra MEIJI
```

The singleton instance for the 'Meiji' era (1868-01-01 - 1912-07-29) which has the value -1.

TAISHO

```
public static final JapaneseEra TAISHO
```

The singleton instance for the 'Taisho' era (1912-07-30 - 1926-12-24) which has the value 0.

SHOWA

```
public static final JapaneseEra SHOWA
```

The singleton instance for the 'Showa' era (1926-12-25 - 1989-01-07) which has the value 1.

HEISEI

```
public static final JapaneseEra HEISEI
```

The singleton instance for the 'Heisei' era (1989-01-08 - 2019-04-30) which has the value 2.

Method Detail

of

```
public static JapaneseEra of(int japaneseEra)
```

Obtains an instance of `JapaneseEra` from an `int` value.

- The value `1` is associated with the 'Showa' era, because it contains 1970-01-01 (ISO calendar system).
- The values `-1` and `0` are associated with two earlier eras, Meiji and Taisho, respectively.
- A value greater than `1` is associated with a later era, beginning with Heisei (2).

Every instance of `JapaneseEra` that is returned from the `values()` method has an int value (available via `Era.getValue()` which is accepted by this method.

Parameters:

`japaneseEra - the era to represent`

Returns:

the JapaneseEra singleton, not null

Throws:

DateTimeException - if the value is invalid

valueOf

```
public static JapaneseEra valueOf(String japaneseEra)
```

Returns the JapaneseEra with the name.

The string must match exactly the name of the era. (Extraneous whitespace characters are not permitted.)

Parameters:

japaneseEra - the japaneseEra name; non-null

Returns:

the JapaneseEra singleton, never null

Throws:

IllegalArgumentException - if there is not JapaneseEra with the specified name

values

```
public static JapaneseEra[] values()
```

Returns an array of JapaneseEras.

This method may be used to iterate over the JapaneseEras as follows:

```
for (JapaneseEra c : JapaneseEra.values())
    System.out.println(c);
```

Returns:

an array of JapaneseEras

getDisplayName

```
public String getDisplayName(TextStyle style,
                             Locale locale)
```

Gets the textual representation of this era.

This returns the textual name used to identify the era, suitable for presentation to the user. The parameters control the style of the returned text and the locale.

If no textual mapping is found then the `numeric value` is returned.

Specified by:

`getDisplayName` in interface `Era`

Parameters:

`style - the style of the text required, not null`

`locale - the locale to use, not null`

Returns:

`the text value of the era, not null`

getValue

`public int getValue()`

Gets the numeric era `int` value.

The `SHOWA` era that contains 1970-01-01 (ISO calendar system) has the value 1. Later eras are numbered from 2 (`HEISEI`). Earlier eras are numbered 0 (`TAISHO`), -1 (`MEIJI`)).

Specified by:

`getValue` in interface `Era`

Returns:

`the era value`

range

`public ValueRange range(TemporalField field)`

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This era is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `ERA` field returns the range. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this`

as the argument. Whether the range can be obtained is determined by the field.

The range of valid Japanese eras can change over time due to the nature of the Japanese calendar system.

Specified by:

`range` in interface `Era`

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field` - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

`DateTimeException` - if the range for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the unit is not supported

toString

```
public String toString()
```

Description copied from class: `Object`

Returns a string representation of the object. In general, the `toString` method returns a string that "textually represents" this object. The result should be a concise but informative representation that is easy for a person to read. It is recommended that all subclasses override this method.

The `toString` method for class `Object` returns a string consisting of the name of the class of which the object is an instance, the at-sign character `@', and the unsigned hexadecimal representation of the hash code of the object. In other words, this method returns a string equal to the value of:

```
getClass().getName() + '@' + Integer.toHexString(hashCode())
```

Overrides:

`toString` in class `Object`

Returns:

```
a string representation of the object.
```

C

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.